

Fixing inconsistencies between `constexpr` and `constexpr` functions

Document Number: P1937R2

Date: 2020-02-14

Author: David Stone (david.stone@uber.com, david@doublewise.net)

Audience: Core Working Group (CWG)

Summary

A `constexpr` function is the same as a `constexpr` function except that:

1. A `constexpr` function is guaranteed to be evaluated at compile time.
2. You cannot have a pointer to a `constexpr` function except in the body of another `constexpr` function.
3. All overrides of a virtual function must be `constexpr` if the base function is `constexpr`, otherwise they must not be `constexpr`.
4. A `constexpr` function is evaluated even in contexts that would otherwise be unevaluated.

The first point is the intended difference between the two functions. The second point is a consequence of the first. The third point (virtual functions) ends up making sense as a difference between them because of what is fundamentally possible. The final point seems like an unfortunate inconsistency in the language. This paper proposes changing the rules surrounding bullet 4 to unify the behavior of `constexpr` and `constexpr` functions for C++20.

Revision history

Since R0

- Removed section on virtual functions: it will be its own paper in the future, targeted at C++23.
- Added a small piece of wording

Unevaluated contexts

Requiring evaluation of `constexpr` functions in unevaluated contexts does not seem necessary. When I [asked about this discrepancy on the core reflector](#), the answer that I got was essentially that evaluation of a `constexpr` function may have side-effects on the abstract machine, and we would

like to preserve these side-effects. However, `constexpr` functions are not special in this ability, so it seems strange to have a rule that makes them special. It seems to me that we should either

1. not have a special case for `constexpr`, which would mean that it is not evaluated in an unevaluated operand, or
2. Rethink the concept of "unevaluated operands".

This paper argues in favor of option 1, as option 2 is a breaking change that doesn't seem to bring much benefit.

My understanding is that the status quo means the following is valid:

```
constexpr auto add1(auto lhs, auto rhs) {  
    return lhs + rhs;  
}  
using T = decltype(add1(std::declval<int>(), std::declval<int>()));
```

but this very similar code is ill-formed

```
constexpr auto add2(auto lhs, auto rhs) {  
    return lhs + rhs;  
}  
using T = decltype(add2(std::declval<int>(), std::declval<int>()));
```

Wording

Modify [expr.const], paragraph 12 as follows:

An expression or conversion is in an immediate function context if it is potentially evaluated and its innermost non-block scope is a function parameter scope of an immediate function. An expression or conversion is an immediate invocation if it is an potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

Modify [expr.prim.id] paragraph 3 as follows:

An potentially-evaluated id-expression that denotes an immediate function ([dcl.constexpr]) shall appear only